Full Stack Inventory Management System

Task 04 - Project Report

Student Name

Hassan Asad

Internship Domain

Web Development

Organization

Devixo Solutions

Task Number

04

Technology Stack

HTML, CSS, JavaScript
Node.js, Express, MongoDB

Report Date

4 November 2026

Final task: Full-Stack Inventory Management System

Project Overview

Objective

Build a private inventory register for a shop owner. After signing in, the owner can record products, update stock, identify shortages and see the value of the inventory in Pakistani rupees (PKR). Records persist after logout and across deployments.

Technologies Used

Technology	Purpose
HTML, CSS and JavaScript	Build the responsive interface, fetch API data and update the dashboard.
Node.js and Express	Handle authentication, product requests and input validation.
MongoDB Atlas / Mongoose	Store records and define schemas and indexes.
bcryptjs	Hash passwords and verify credentials at login.
Session management	express-session manages cookies; connect-mongo stores sessions in MongoDB.
Vercel	Serve static files and run the exported Express application.

Website / Application Structure

public/index.html and style.css - Forms, navigation, tables and responsive themes.

public/app.js - Dashboard rendering, API calls, filters, pagination and editing.

server.js and db.js - Express setup, sessions, database connection and error handling.

routes/auth.js and routes/products.js - Account access and inventory operations.

models/ - User, Product and Activity schemas.

Application Flow

The browser sends requests to Express. For protected requests, Express checks the session and uses its account ID to query Atlas. The response updates the register, overview and recent activity.

Implementation and Features

Implemented Features

Module	Implemented behavior
Authentication	Register, log in and log out. A server-side session identifies the account.
Product management	Create, edit or delete a product; confirm deletion before the request is sent.
Dashboard	Show product count, total units, stock value and products that need restocking.
Stock availability	Separate in-stock, low-stock and empty products; open the restocking list directly.
Search and pagination (bonus)	Filter by name, SKU, category or availability. Sort the matches and show ten products per page.
Activity and interface	Show the ten most recent changes. Section navigation, mobile layout and saved light/dark preferences support daily use.

Database and Product Fields

Products store the owner ID, name, SKU, category, quantity, unit price and timestamps. SKUs are normalized to uppercase. A compound unique index on owner and SKU prevents duplicates within one account while allowing separate accounts to use the same SKU.

Users store a name, normalized email and password hash. Activity entries record the owner, product name, action and creation time. Session records are stored separately in MongoDB.

Validation and Access Rules

Product names, SKUs and categories are required. Quantity must be a non-negative integer, and price must be a finite, non-negative number. The API validates these values even when a request bypasses the browser form.

Registration checks email format and requires a password of at least eight characters within the 72-byte bcrypt limit. Private routes reject signed-out requests. Updates and deletions match both the product ID and the current owner.

Authentication Screens

Figure 1. Login screen

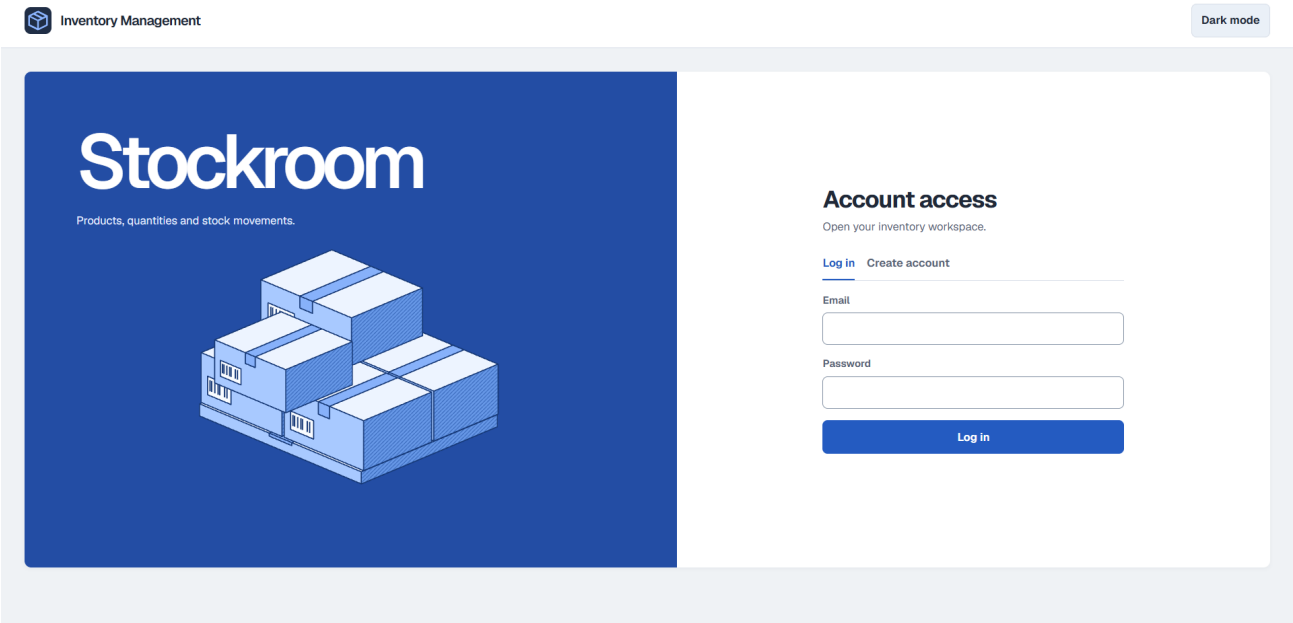
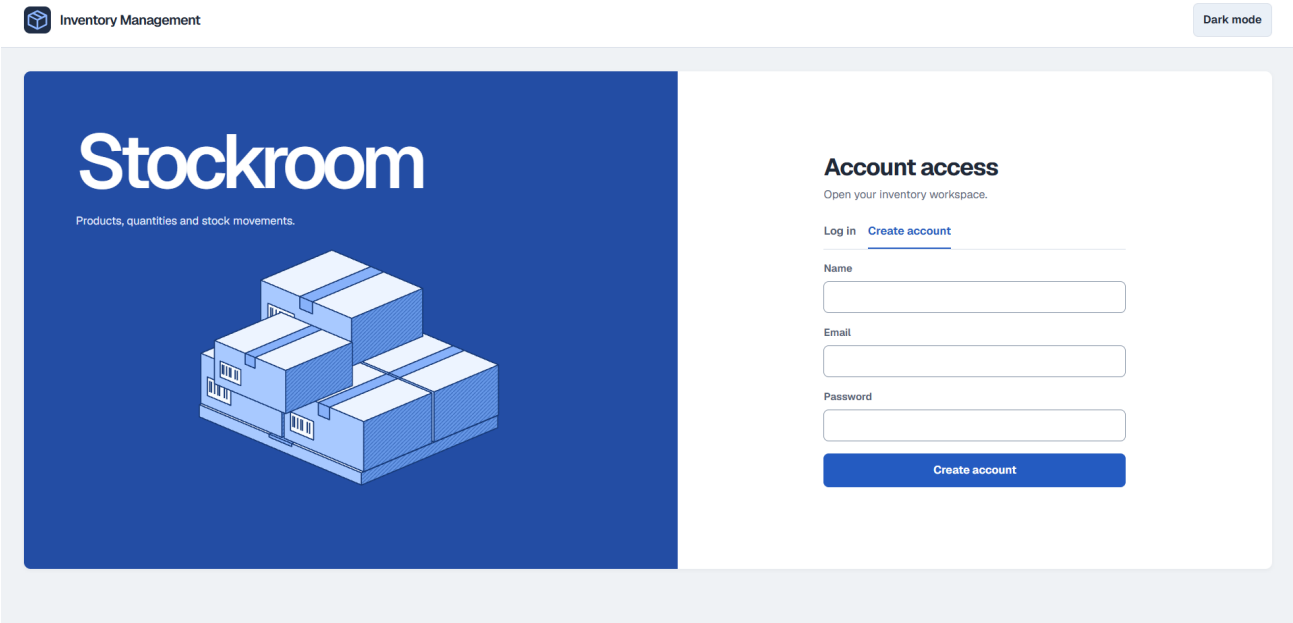


Figure 2. Registration screen



Registration creates a private inventory workspace. Login verifies the password hash and starts a server-side session. Logout destroys the session and clears the cookie.

Dashboard and Stock Availability

Figure 3. Stock overview

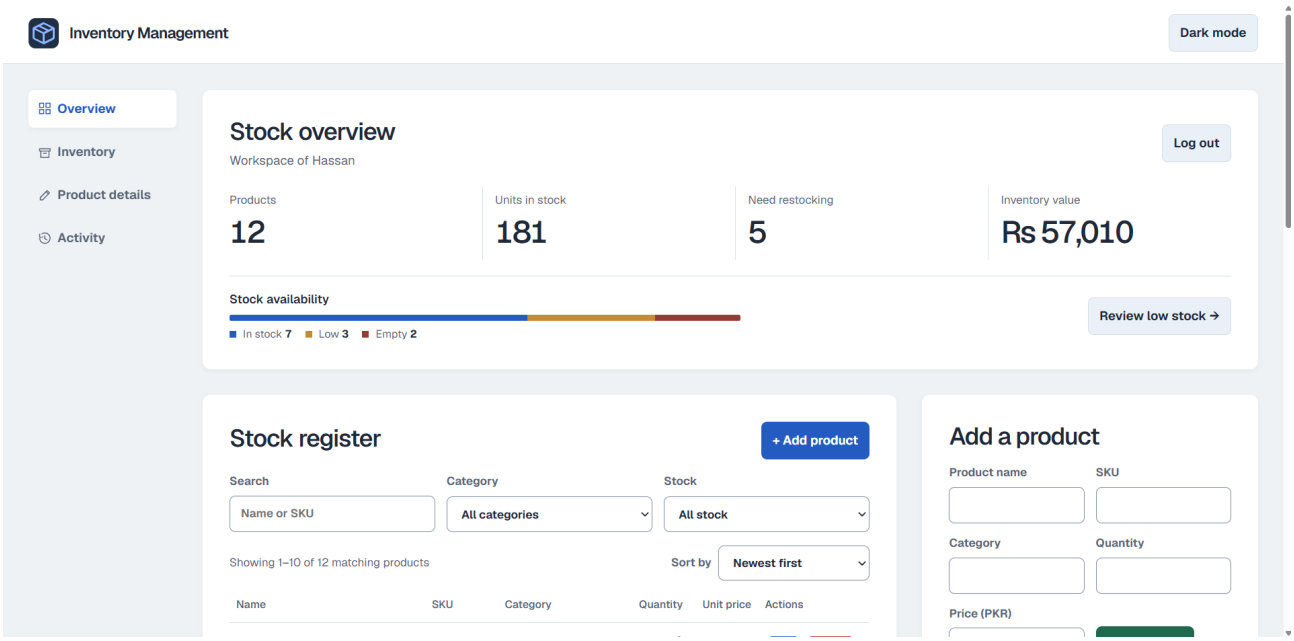
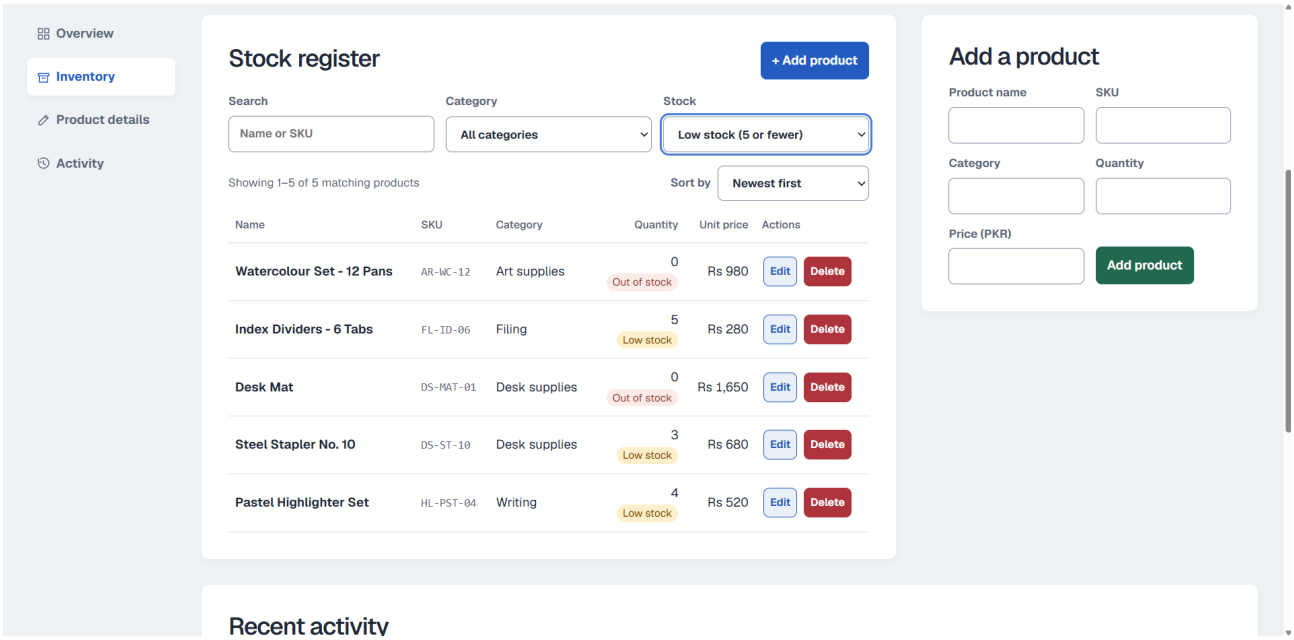


Figure 4. Products needing restocking



Inventory value is the sum of quantity multiplied by unit price for all products. Products with one to five units are low; products with zero units are empty. Both groups appear in the restocking view.

Stock Register and Pagination

Figure 5a. Register, filters and page controls

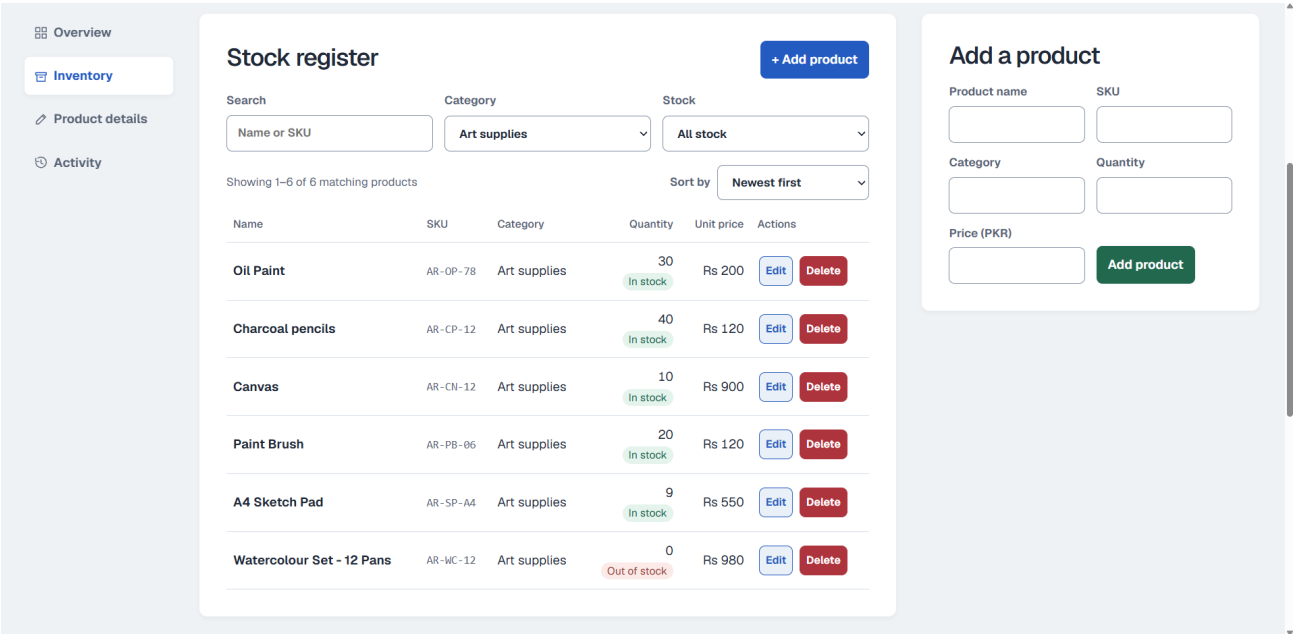
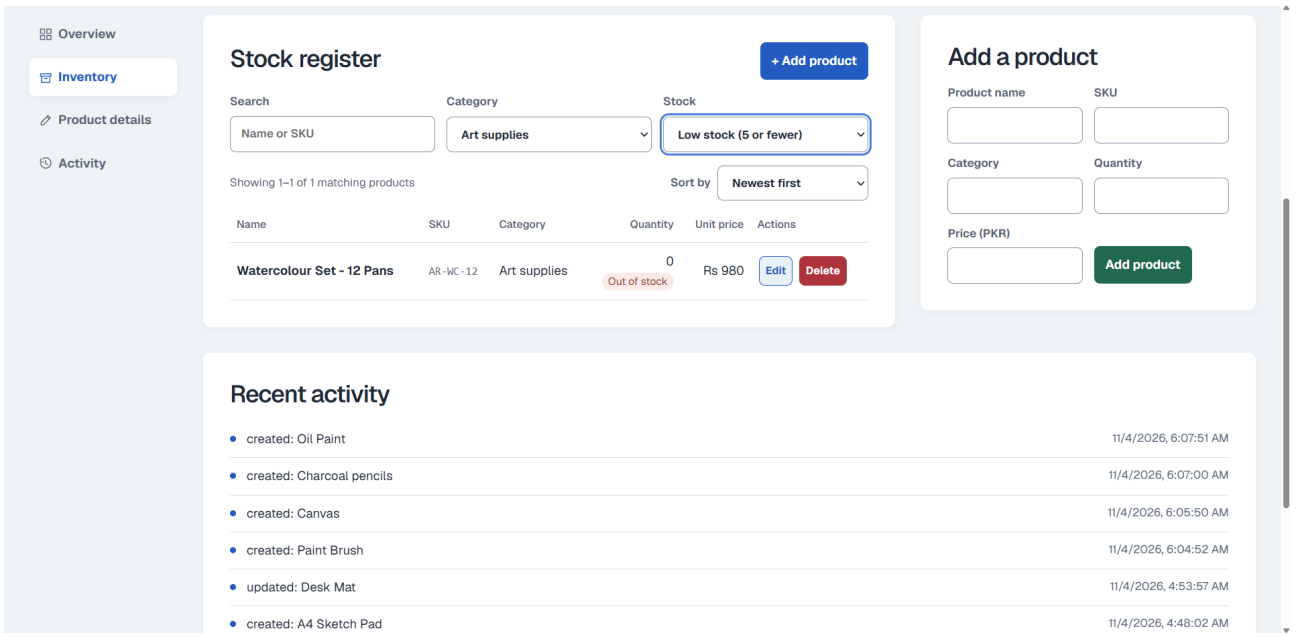


Figure 5b. Search or category filter



Search matches a product name or SKU. Filtering and sorting happen before pagination, so a search covers all matching records. Dashboard totals still describe the entire account.

Product Operations

Figure 6. Add product

Name	SKU	Category	Quantity	Unit price	Actions
Watercolour Set - 12 Pans	AR-WC-12	Art supplies	0 Out of stock	Rs 980	Edit Delete
Index Dividers - 6 Tabs	FL-ID-06	Filing	5 Low stock	Rs 280	Edit Delete
Desk Mat	DS-MAT-01	Desk supplies	0 Out of stock	Rs 1,650	Edit Delete
Steel Stapler No. 10	DS-ST-10	Desk supplies	3 Low stock	Rs 680	Edit Delete
Pastel Highlighter Set	HL-PST-04	Writing	4 Low stock	Rs 520	Edit Delete

Figure 7. Edit product

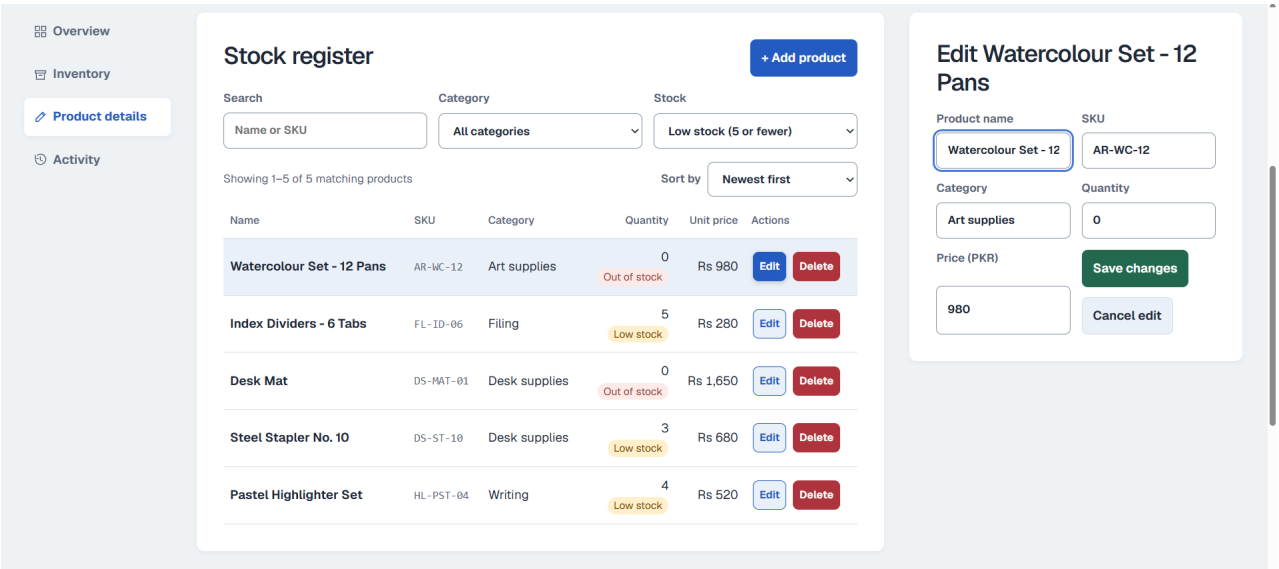
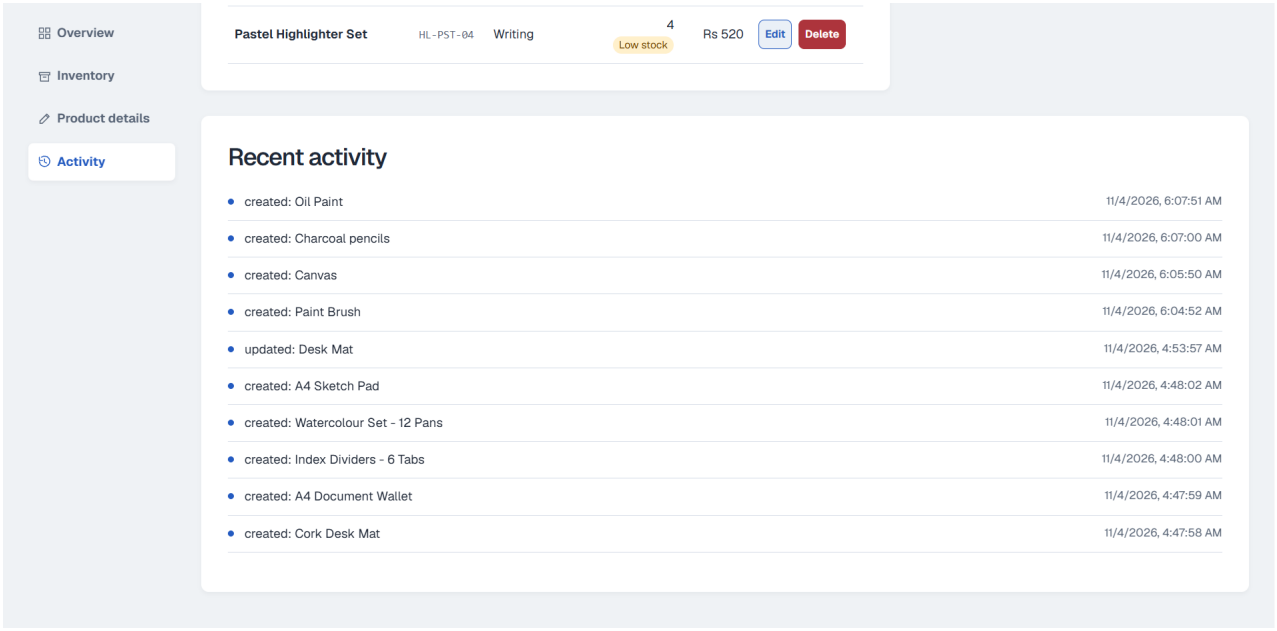


Figure 8. Recent activity



The editor opens on the selected product. Saving refreshes the table and records the action; canceling returns to the register without changing the product.

Mobile Layout and Themes

Figure 9. Mobile - light mode

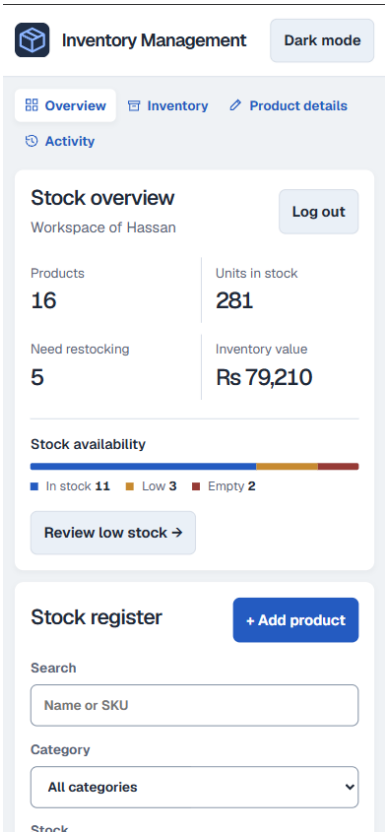
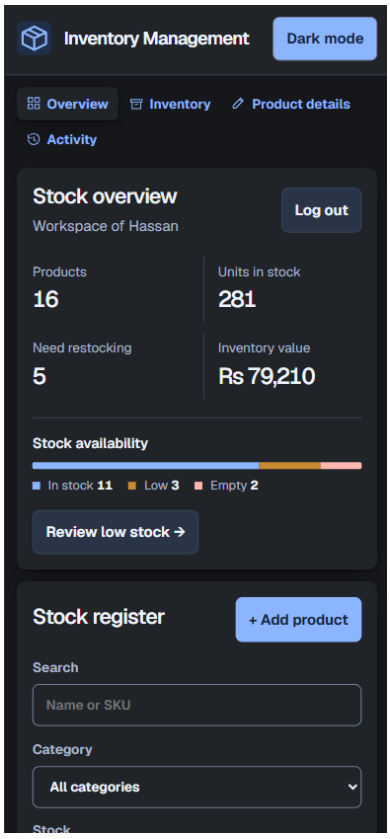


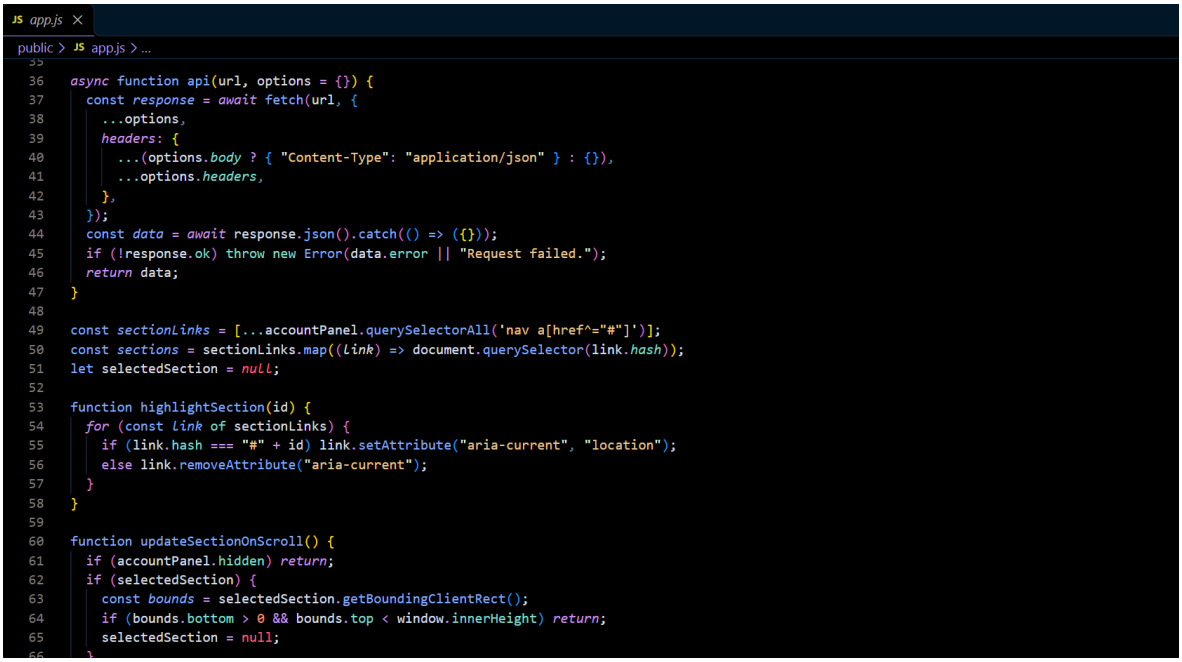
Figure 10. Mobile - dark mode



On narrow screens, the dashboard sections stack vertically and the table scrolls within its container. The theme preference is saved in the browser. Stock badges include text so users can distinguish low and empty products in either theme.

Code Screenshots

Figure 11. Frontend API request and error handling



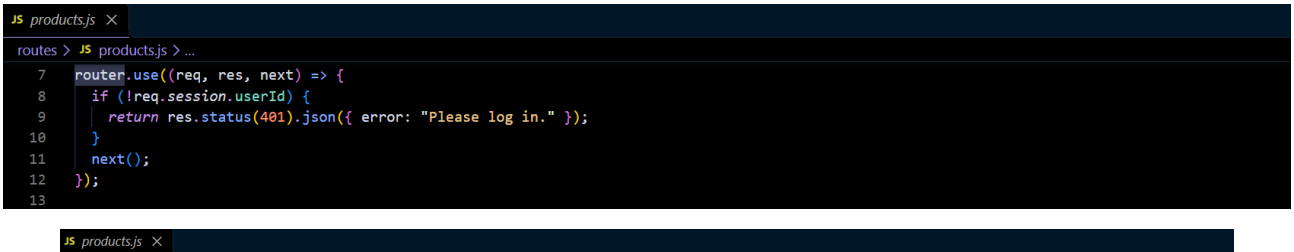
```
app.js
public > app.js > ...
async function api(url, options = {}) {
  const response = await fetch(url, {
    ...options,
    headers: {
      ...(options.body ? { "Content-Type": "application/json" } : {}),
      ...options.headers,
    },
  });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data.error || "Request failed.");
  return data;
}

const sectionLinks = [...accountPanel.querySelectorAll('nav a[href^="#"]')];
const sections = sectionLinks.map((link) => document.querySelector(link.hash));
let selectedSection = null;

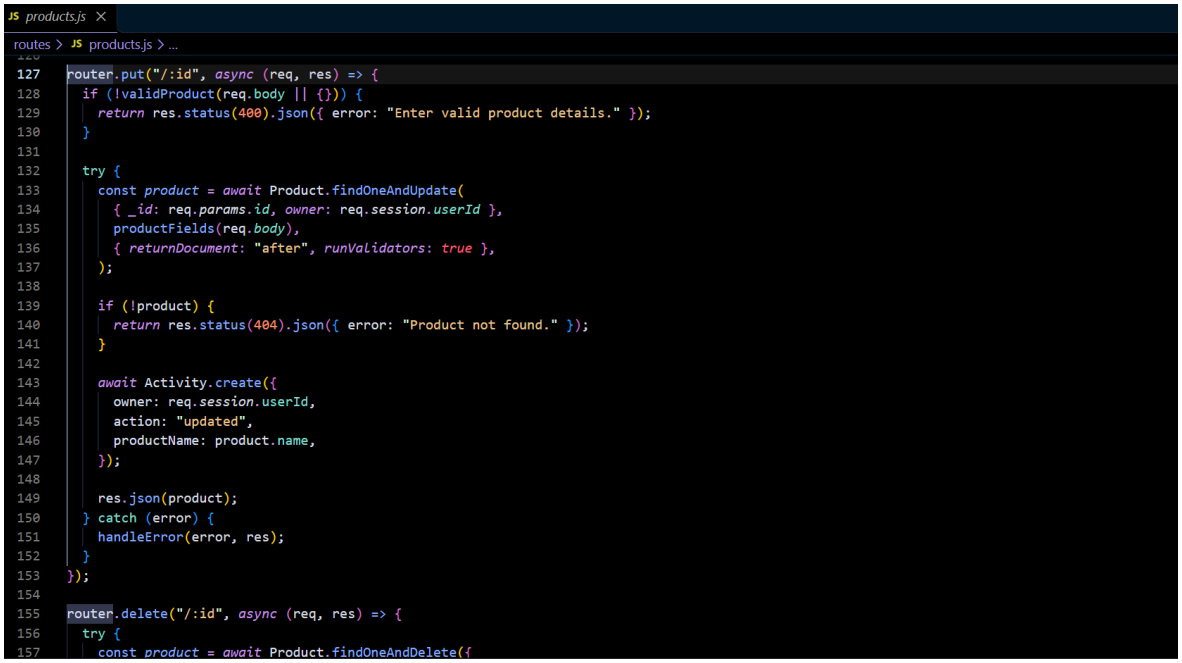
function highlightSection(id) {
  for (const link of sectionLinks) {
    if (link.hash === "#" + id) link.setAttribute("aria-current", "location");
    else link.removeAttribute("aria-current");
  }
}

function updateSectionOnScroll() {
  if (accountPanel.hidden) return;
  if (selectedSection) {
    const bounds = selectedSection.getBoundingClientRect();
    if (bounds.bottom > 0 && bounds.top < window.innerHeight) return;
    selectedSection = null;
```

Figure 12a. Protected product routes



```
products.js
routes > products.js > ...
router.use((req, res, next) => {
  if (!req.session.userId) {
    return res.status(401).json({ error: "Please log in." });
  }
  next();
});
```



```
products.js
routes > products.js > ...
router.put("/:id", async (req, res) => {
  if (!validProduct(req.body || {})) {
    return res.status(400).json({ error: "Enter valid product details." });
  }

  try {
    const product = await Product.findOneAndUpdate(
      { _id: req.params.id, owner: req.session.userId },
      productFields(req.body),
      { returnDocument: "after", runValidators: true },
    );

    if (!product) {
      return res.status(404).json({ error: "Product not found." });
    }

    await Activity.create({
      owner: req.session.userId,
      action: "updated",
      productName: product.name,
    });

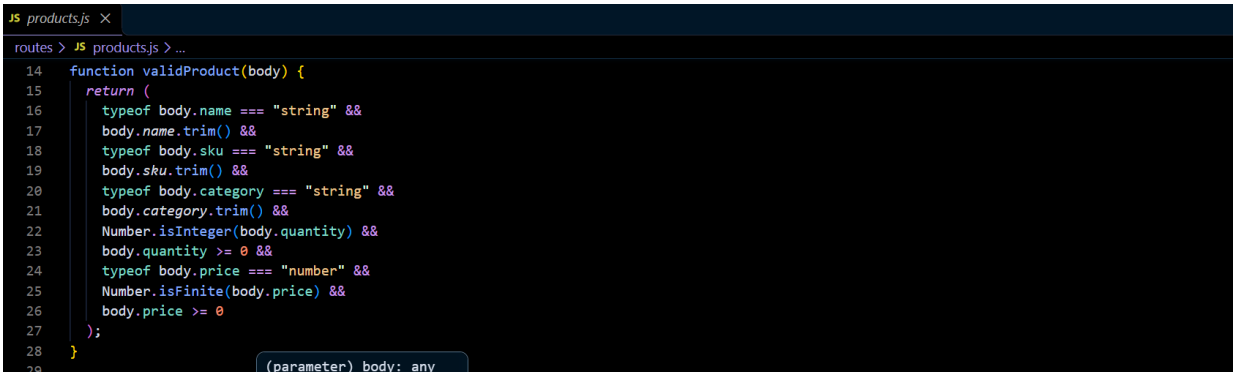
    res.json(product);
  } catch (error) {
    handleError(error, res);
  }
});

router.delete("/:id", async (req, res) => {
  try {
    const product = await Product.findOneAndDelete({
```

The frontend sends JSON to the API and displays error messages when requests fail. The backend obtains the owner ID from the session and applies it when reading or modifying records.

Validation and Conclusion

Figure 12b. Product field validation



Testing Summary

Chrome tests exercised registration, login, logout, product creation and editing, cancellation, deletion, search, category and stock filters, sorting, pagination and saved theme preferences. Checks included deleting the last product on a page and refreshing after an edit.

API checks confirmed that signed-out requests return HTTP 401 and malformed registration or login inputs return HTTP 400. The deployed homepage and health endpoint returned HTTP 200. Local checks used a temporary account and disposable records.

Deployment and Conclusion

Vercel hosts the application and Atlas persists inventory and sessions. The production MongoDB URI and session secret are configured as environment variables. The project meets the core assignment requirements and implements searchable pagination as its bonus.

Live Application

https://inventory-management-one-sand.vercel.app/

GitHub Repository

https://github.com/HassanAsad118/Inventory-management